WEEK 1

Kotlin Basics: Your First Programs

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

This week you will learn the building blocks of every Kotlin program: printing text, storing values in variables, doing calculations, making decisions, repeating work with loops, and organising code into functions. At the end of the week you will build your first real project: a **calculator**.

Day	Topic	You will be able to...
Day 1	Introduction & your first program	Write and run a Kotlin program, print text, use comments
Day 2	Variables & data types	Store values with val/var, use Int, Double, String, Boolean, Char
Day 3	Operators, conversion & input	Do maths, compare values, convert types, read user input
Day 4	Conditions: if & when	Make your program take decisions
Day 5	Loops	Repeat work with for, while, do-while, break, continue
Day 6	Functions + weekly project	Write reusable functions and build a calculator

Where to write your code this week

Kotlin Playground (https://play.kotlinlang.org) runs in your browser, so you don't need to install anything. Use it for Days 1, 2, 4, 5 and 6.

IntelliJ IDEA Community Edition (free, from jetbrains.com/idea) is needed only for programs that **read input from the keyboard** (Day 3 and the Day 6 project), because the Playground cannot read input. Install it on Day 3.

How to use each day's lesson

Hour	What to do
Hour 1: Learn	Read the lesson slowly. Understand each example before moving on.
Hour 2: Practice	**Type** every example yourself (don't copy-paste) and run it. Then change values and see what happens.
Hour 3: Build	Solve the Practice Exercises **without** looking at the answers.
Hour 4: Revise	Answer the Quick Quiz, check the Answer Key at the end, and save your code.

> **Tip:** Every example in this book has been run, and the output shown is the real output. If your output is different, compare your code letter by letter. A small typo is usually the reason.

WEEK 1 • DAY 1

Introduction & Your First Program

Today's goal: Understand what Kotlin is, run your first program, and print text on the screen.

1. What is Kotlin?

Kotlin is a modern programming language created by **JetBrains** in 2011. In 2019 Google announced that Android development is **Kotlin-first**, so most new Android apps and Google's own examples use Kotlin.

- **Easy to read**: less code than Java for the same work.
- **Safe**: it prevents many common errors (like the famous "null pointer" crash).
- **Used everywhere**: Android apps, servers, desktop apps, and even iOS (Kotlin Multiplatform).

2. Open the Kotlin Playground

- Open **https://play.kotlinlang.org** in your browser.
- You will see some sample code. Delete it.
- Type the program below, then click the purple **Run** button.
- The output appears at the bottom of the screen.

3. Your first program

Hello World

```
fun main() {
    println("Hello, World!")
}
```

Output

```
Hello, World!
```

Let's understand every part:

Code	Meaning
fun	Short for **function**. A function is a block of code with a name.
main	The name of the function. **main** is special: Kotlin always starts running your program here.
( )	Brackets for inputs (parameters). main has none here.
{ }	Curly braces mark the start and end of the function's body.
println(...)	**Print line**: shows text on the screen, then moves to a new line.
"Hello, World!"	Text inside double quotes is called a **String**.

Tip: Kotlin runs the lines inside main from **top to bottom**, one by one.

4. println vs print

println prints and then moves to the next line. **print** prints but stays on the same line.

print and println

```
fun main() {
    print("Hello, ")
    print("Anjana")
    println()
    println("Welcome to Kotlin!")
    println("Let's learn Android.")
}
```

Output

```
Hello, Anjana
Welcome to Kotlin!
Let's learn Android.
```

An empty **println()** just moves to a new line.

5. Escape characters

Some characters are special inside a String. You write them with a backslash (\):

Escape	Meaning	Example
\n	New line	"Line 1\nLine 2"
\t	Tab (big space)	"Name:\tAnjana"
\"	Double quote inside a String	"She said \"Hi\""
\\	A backslash	"C:\\Users"
\$	A dollar sign	"Price: \$10"

Escape characters

```
fun main() {
    println("Line 1\nLine 2")
    println("Name:\tAnjana")
    println("She said, \"Kotlin is fun!\"")
    println("Path: C:\\Users\\Anjana")
}
```

Output

```
Line 1
Line 2
Name:   Anjana
She said, "Kotlin is fun!"
Path: C:\Users\Anjana
```

6. Comments

Comments are notes for humans. Kotlin **ignores** them completely. Use them to explain your code.

Comments

```
fun main() {
    // This is a single-line comment
    println("Comments are ignored by Kotlin")

    /* This is a
       multi-line comment */
    println("Only this code runs") // comment at the end of a line
}
```

Output

```
Comments are ignored by Kotlin
Only this code runs
```

7. A complete example: My Profile

My Profile

```
fun main() {
    println("===== My Profile =====")
    println("Name    : Anjana")
    println("Course  : Android Development")
    println("Language: Kotlin")
    println("Goal    : Become an Android Developer")
    println("======================")
}
```

Output

```
===== My Profile =====
Name    : Anjana
Course  : Android Development
Language: Kotlin
Goal    : Become an Android Developer
======================
```

8. Common mistakes

Wrong code	Problem	Fix
Println("Hi")	Kotlin is **case-sensitive**. Println is not println.	println("Hi")
println("Hi)	The closing quote is missing.	println("Hi")
println(Hi)	Text must be inside quotes.	println("Hi")
fun main() { ... (no })	Every { needs a matching }.	Add the closing }
fun Main()	The function must be called main (lowercase).	fun main()

Tip: When you see an error, read the **line number** in the message. The mistake is on that line or just before it.

9. Save your work (GitHub)

- Create a free account at **github.com**.
- Click **New repository** and name it **kotlin-practice**.
- Every day, save your programs as files (for example day1_hello.kt) and upload them with **Add file** → **Upload files**. You'll learn proper Git commands in Week 3.

Practice Exercises

1. Print your name, your college and your city on three separate lines using three println statements.

2. Print the same three lines using only **one** println and \n.

3. Print this pattern:

```
*
**
***
****
```

4. Print this sentence exactly, including the quotes: `Anjana says "I love coding!"`

5. Print a small marks table using \t: a header line *Subject, Marks*, then *Maths 90*, *Physics 85*, *English 88*.

6. Add comments to your profile program that explain what each line does.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Where does every Kotlin program start running?

Q2. What is the difference between print and println?

Q3. Which escape character creates a new line?

Q4. Are println and Println the same?

Q5. What does Kotlin do with comments?

WEEK 1 • DAY 2

Variables & Data Types

Today's goal: Store information in variables, understand val vs var, learn the basic data types and string templates.

1. What is a variable?

A variable is like a **labelled box** that stores a value. The label is the variable's **name**, and the thing inside is its **value**. Kotlin has two kinds of variables:

Keyword	Meaning	Example
val	**Value**: read-only. Once set, it can **never** change.	val name = "Anjana"
var	**Variable**: can be changed later.	var age = 20

val and var

```
fun main() {
    val name = "Anjana"     // val: cannot change
    var age = 20            // var: can change

    println("Name: " + name)
    println("Age: " + age)

    age = 21                // OK, because age is a var
    println("Next year age: " + age)
}
```

Output

```
Name: Anjana
Age: 20
Next year age: 21
```

This gives an ERROR

```
val name = "Anjana"
name = "Anju"      // Error: Val cannot be reassigned
```

Tip: Golden rule: always use **val** first. Change it to **var** only if the value really needs to change. Professional Android code uses val most of the time.

2. Basic data types

Every value has a **type** that tells Kotlin what kind of data it is:

Type	Stores	Example
Int	Whole numbers (up to about 2.1 billion)	val age = 20
Long	Very big whole numbers (add L)	val population = 8_000_000_000L
Double	Decimal numbers (most common)	val price = 99.50
Float	Decimal numbers, less precise (add f)	val weight = 55.5f
Boolean	Only true or false	val isStudent = true
Char	A single character, in **single** quotes	val grade = 'A'

Type	Stores	Example
String	Text, in **double** quotes	val city = "Kochi"

All the data types

```
fun main() {
    val age: Int = 20
    val population: Long = 8_000_000_000L
    val price: Double = 99.50
    val weight: Float = 55.5f
    val isStudent: Boolean = true
    val grade: Char = 'A'
    val city: String = "Kochi"

    println(age)
    println(population)
    println(price)
    println(weight)
    println(isStudent)
    println(grade)
    println(city)
}
```

Output

```
20
8000000000
99.5
55.5
true
A
Kochi
```

The underscores in **8_000_000_000** are only there to make big numbers easier to read. Kotlin ignores them.

3. Type inference

In the example above we wrote the type after a colon (**val age: Int = 20**). Usually you don't need to, because Kotlin **guesses the type** from the value. This is called **type inference**.

Kotlin guesses the type

```
fun main() {
    val a = 10        // Int
    val b = 10.5      // Double
    val c = "Hi"      // String
    val d = true      // Boolean
    val e = 'K'       // Char

    println(a::class.simpleName)
    println(b::class.simpleName)
    println(c::class.simpleName)
    println(d::class.simpleName)
    println(e::class.simpleName)
}
```

Output

```
Int
Double
String
Boolean
Char
```

(**::class.simpleName** just shows the type's name. You don't need to remember it.)

Once a variable has a type, it **keeps** that type:

This gives an ERROR

```
var age = 20        // age is an Int
age = "twenty"      // Error: type mismatch (String is not Int)
```

4. Naming rules

Rule	Good	Bad
Use camelCase: first word small, next words start with a capital	studentName	student_name, StudentName
Can't start with a number	subject1	1subject
No spaces or symbols like - or @	totalMarks	total marks, total-marks
Can't use keywords	myValue	val, fun, if
Use meaningful names	price	p, x1

5. String templates

Instead of joining strings with +, you can put variables **inside** a String with **$**. For a calculation or expression, use **${ }**.

String templates

```
fun main() {
    val name = "Anjana"
    val age = 20

    println("My name is $name and I am $age years old.")
    println("Next year I will be ${age + 1}.")
    println("My name has ${name.length} letters.")
}
```

Output

```
My name is Anjana and I am 20 years old.
Next year I will be 21.
My name has 6 letters.
```

6. Useful String functions

Working with Strings

```
fun main() {
    val language = "Kotlin"

    println(language.length)            // number of characters
    println(language.uppercase())       // all capital letters
    println(language.lowercase())       // all small letters
    println(language[0])                // first character (counting starts at 0)
    println(language[language.length - 1])  // last character
    println(language.reversed())        // backwards
    println(language.contains("lin"))   // is "lin" inside it?
}
```

Output

```
6
KOTLIN
kotlin
K
n
niltoK
true
```

Careful: Positions in a String start at **0**, not 1. In "Kotlin", K is at position 0 and n is at position 5.

7. Complete example: Student Report

Student Report

```
fun main() {
    val studentName = "Anjana"
    val rollNumber = 25
    val marks = 456.5
    val totalMarks = 500
    val passed = true

    val percentage = marks * 100 / totalMarks

    println("Student : $studentName")
    println("Roll No : $rollNumber")
    println("Marks   : $marks / $totalMarks")
    println("Percent : $percentage%")
    println("Passed  : $passed")
}
```

Output

```
Student : Anjana
Roll No : 25
Marks   : 456.5 / 500
Percent : 91.3%
Passed  : true
```

Practice Exercises

1. Create val variables for your name and college, and a var for your current semester. Print them. Then change the semester and print it again.

2. Create one variable of each type: Int, Long, Double, Boolean, Char and String. Print each one.

3. Using a string template, print: *Hello, my name is ___. I am ___ years old and I live in ___.*

4. Store the word "Android" in a val. Print its length, uppercase version, first letter, last letter and reversed version.

5. Find and fix the error: `val pi = 3.14` then `pi = 3.14159`

6. Make a **Book Card** program: title, author, price (Double), pages (Int), isAvailable (Boolean). Print it neatly using string templates.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between val and var?

Q2. Which one should you use by default?

Q3. Which type would you use for 9.99?

Q4. Which quotes are used for a Char and which for a String?

Q5. What does println("Sum: ${2 + 3}") print?

Q6. What is the type of val big = 100L ?

WEEK 1 • DAY 3

Operators, Type Conversion & User Input

Today's goal: Do calculations, compare values, combine conditions, convert between types and read input from the keyboard.

1. Arithmetic operators

Operator	Meaning	Example	Result
+	Add	17 + 5	22
-	Subtract	17 - 5	12
*	Multiply	17 * 5	85
/	Divide	17 / 5	3 (Int division!)
%	Remainder (modulus)	17 % 5	2

Arithmetic

```
fun main() {
    val a = 17
    val b = 5

    println("a + b = ${a + b}")
    println("a - b = ${a - b}")
    println("a * b = ${a * b}")
    println("a / b = ${a / b}")   // whole-number division
    println("a % b = ${a % b}")   // remainder
}
```

Output

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
```

2. Integer division: a very common trap

When you divide an **Int by an Int**, Kotlin throws away the decimal part. If you want the decimal answer, at least one number must be a Double.

Int vs Double division

```
fun main() {
    println(7 / 2)              // Int / Int = Int
    println(7.0 / 2)            // a Double is involved
    println(7 / 2.0)
    println(7.toDouble() / 2)   // convert first
}
```

Output

```
3
3.5
3.5
3.5
```

Tip: % is very useful: **n % 2 == 0** means n is even, and **n % 10** gives the last digit of n.

3. Assignment operators and ++ / --

Short-cut operators

```
fun main() {
    var score = 10
    score += 5    // same as score = score + 5
    println(score)
    score -= 3    // score = score - 3
    println(score)
    score *= 2    // score = score * 2
    println(score)
    score /= 4    // score = score / 4
    println(score)
    score %= 4    // score = score % 4
    println(score)
    score++       // add 1
    println(score)
    score--       // subtract 1
    println(score)
}
```

Output

```
15
12
24
6
2
3
2
```

4. Comparison and logical operators

Comparison operators give a **Boolean** (true or false):

Operator	Meaning	Operator	Meaning
==	equal to	!=	not equal to
>	greater than	<	less than
>=	greater than or equal	<=	less than or equal

Logical operators combine Booleans:

Operator	Name	Rule
&&	AND	true only if **both** sides are true
||	OR	true if **at least one** side is true
!	NOT	flips: !true is false, !false is true

Comparison and logic

```
fun main() {
    val age = 20
    val hasId = true

    println(age > 18)           // true
    println(age == 21)          // false
    println(age != 21)          // true
    println(age >= 20)          // true
    println(age > 18 && hasId)  // both true -> true
    println(age < 18 || hasId)  // one true  -> true
    println(!hasId)             // opposite  -> false
}
```

Output

```
true
false
true
true
true
true
false
```

Careful: = stores a value (age = 20). **==** compares two values (age == 20). Mixing them up is a very common mistake.

5. Type conversion

Sometimes you need to change a value from one type to another, for example text "25" into the number 25.

Converting types

```
fun main() {
    val text = "25"
    val number = text.toInt()
    println(number + 5)          // maths: 30
    println(text + 5)            // joins text: 255

    val price = 99.99
    println(price.toInt())       // decimal part removed

    val count = 7
    println(count.toDouble())
    println(count.toString() + " apples")

    val bad = "abc".toIntOrNull()
    println(bad)                 // not a number -> null
}
```

Output

```
30
255
99
7.0
7 apples
null
```

toInt() crashes if the text is not a number. **toIntOrNull()** safely gives **null** (meaning "no value") instead. You'll learn about null in Week 2.

6. Built-in maths functions

Maths helpers

```
import kotlin.math.abs
import kotlin.math.pow
import kotlin.math.sqrt

fun main() {
    println(sqrt(16.0))       // square root
    println(2.0.pow(3))       // 2 to the power 3
    println(abs(-7))          // remove the minus sign
    println(maxOf(10, 25))    // bigger value
    println(minOf(10, 25))    // smaller value
}
```

Output

```
4.0
8.0
7
25
10
```

7. Reading input from the keyboard

> **Careful:** The Kotlin Playground **cannot** read input. Install **IntelliJ IDEA Community Edition** today: **New Project** → **Kotlin**, then write your code in **Main.kt** and click the green Run button.

readln() waits for the user to type a line and press Enter. It always gives a **String**, so convert it if you need a number.

Reading input (run in IntelliJ: input typed by the user is shown in the output)

```
fun main() {
    print("Enter your name: ")
    val name = readln()

    print("Enter your age: ")
    val age = readln().toInt()

    println("Hello $name! In 5 years you will be ${age + 5}.")
}
```

Output

```
Enter your name: Anjana
Enter your age: 20
Hello Anjana! In 5 years you will be 25.
```

8. Complete example: three real-life calculations

Real-life formulas

```
fun main() {
    // 1. Rectangle
    val length = 12
    val width = 5
    println("Area = ${length * width}")
    println("Perimeter = ${2 * (length + width)}")

    // 2. Simple interest: SI = P * R * T / 100
    val principal = 10000.0
    val rate = 7.5
    val years = 2
    val interest = principal * rate * years / 100
    println("Simple Interest = $interest")

    // 3. Celsius to Fahrenheit: F = C * 9 / 5 + 32
    val celsius = 37.0
    val fahrenheit = celsius * 9 / 5 + 32
    println("$celsius C = $fahrenheit F")
}
```

Output

```
Area = 60
Perimeter = 34
Simple Interest = 1500.0
37.0 C = 98.6 F
```

Practice Exercises

1. With a = 25 and b = 4, print the sum, difference, product, Int quotient, remainder and decimal quotient.

2. Find the average of three marks (78, 85, 92) as a Double.

3. Check whether the year 2024 is divisible by 4 using % and print the Boolean result.

4. Convert 5 km into metres and into miles (1 km = 0.621371 miles).

5. (IntelliJ) Ask the user for two numbers and print their sum.

6. Predict the output first, then run it to check: `println(10 / 3)`, `println(10 % 3)`, `println(10.0 / 4)`, `println("10" + 3)`, `println(5 > 3 && 2 > 4)`

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does the % operator give?

Q2. What is the result of 9 / 2 in Kotlin?

Q3. How do you convert the String "45" to an Int?

Q4. When is a && b true?

Q5. What type does readln() return?

Q6. What is the difference between = and == ?

WEEK 1 • DAY 4

Making Decisions: if, else and when

Today's goal: Write programs that choose what to do based on conditions.

1. The if statement

if runs a block of code only when a condition is **true**.

Simple if

```
fun main() {
    val marks = 72

    if (marks >= 40) {
        println("You passed!")
    }
    println("Program finished")
}
```

Output

```
You passed!
Program finished
```

2. if ... else

else runs when the condition is **false**.

Even or odd

```
fun main() {
    val number = 7

    if (number % 2 == 0) {
        println("$number is even")
    } else {
        println("$number is odd")
    }
}
```

Output

```
7 is odd
```

3. else if: many choices

Kotlin checks the conditions **from top to bottom** and runs only the **first** one that is true.

Grade calculator

```
fun main() {
    val marks = 83

    if (marks >= 90) {
        println("Grade A+")
    } else if (marks >= 80) {
        println("Grade A")
    } else if (marks >= 70) {
        println("Grade B")
    } else if (marks >= 40) {
        println("Grade C")
    } else {
        println("Failed")
    }
}
```

Output

```
Grade A
```

83 is not >= 90, so the first block is skipped. 83 >= 80 is true, so it prints "Grade A" and skips all the remaining checks.

4. if as an expression

In Kotlin, **if** can give back a value that you store in a variable. When you use it this way, the **else** part is required.

if returns a value

```
fun main() {
    val a = 15
    val b = 22
    val bigger = if (a > b) a else b
    println("Bigger number is $bigger")

    val age = 17
    val status = if (age >= 18) "Adult" else "Minor"
    println(status)
}
```

Output

```
Bigger number is 22
Minor
```

5. Combining conditions with && and ||

Login check

```
fun main() {
    val username = "anjana"
    val password = "kotlin123"

    if (username == "anjana" && password == "kotlin123") {
        println("Login successful")
    } else {
        println("Wrong username or password")
    }
}
```

Output

```
Login successful
```

Largest of three numbers

```
fun main() {
    val x = 12
    val y = 45
    val z = 30

    if (x >= y && x >= z) {
        println("Largest: $x")
    } else if (y >= x && y >= z) {
        println("Largest: $y")
    } else {
        println("Largest: $z")
    }
}
```

Output

```
Largest: 45
```

6. The when expression

When you compare one value against many options, **when** is cleaner than a long if-else chain. It works like *switch* in other languages.

Day of the week

```
fun main() {
    val day = 3

    when (day) {
        1 -> println("Monday")
        2 -> println("Tuesday")
        3 -> println("Wednesday")
        4 -> println("Thursday")
        5 -> println("Friday")
        6, 7 -> println("Weekend!")
        else -> println("Invalid day")
    }
}
```

Output

```
Wednesday
```

6, 7 -> means "if the value is 6 *or* 7". **else** handles every other value.

7. when with ranges, returning a value

in 90..100 means "between 90 and 100, including both".

Grade with when

```
fun main() {
    val marks = 67

    val grade = when (marks) {
        in 90..100 -> "A+"
        in 80..89 -> "A"
        in 70..79 -> "B"
        in 40..69 -> "C"
        else -> "Fail"
    }
    println("Marks: $marks, Grade: $grade")
}
```

Output

```
Marks: 67, Grade: C
```

8. when without a value

If you write **when** with no value in brackets, each branch can be any condition, just like if-else.

Weather advice

```
fun main() {
    val temperature = 34

    when {
        temperature > 35 -> println("Very hot! Stay inside.")
        temperature > 25 -> println("Warm day.")
        temperature > 15 -> println("Pleasant weather.")
        else -> println("Cold! Wear a jacket.")
    }
}
```

Output

```
Warm day.
```

when with Strings

```
fun main() {
    val signal = "yellow"

    val action = when (signal) {
        "red" -> "Stop"
        "yellow" -> "Get ready"
        "green" -> "Go"
        else -> "Unknown signal"
    }
    println(action)
}
```

Output

```
Get ready
```

9. Common mistakes

ERROR: = instead of ==

```
if (marks = 40) { }          // Error: = stores, it does not compare
if (marks == 40) { }         // Correct
```

ERROR: missing else in an if expression

```
val result = if (a > b) a            // Error: 'if' must have an 'else' branch
val result = if (a > b) a else b     // Correct
```

Tip: Use **if** for one or two conditions. Use **when** when there are three or more choices. It's easier to read.

Practice Exercises

1. Check whether a number is **positive**, **negative** or **zero**.

2. Leap year: a year is a leap year if it is divisible by 4 and not by 100, OR divisible by 400. Test it with 2024, 1900 and 2000.

3. Electricity bill: up to 100 units costs Rs 3 per unit, 101 to 200 units costs Rs 5 per unit, and above 200 units costs Rs 8 per unit (apply that rate to all units). Print the bill for 150 units.

4. Using **when**, convert a month number (1 to 12) into the month name.

5. Using **when**, check whether a letter is a vowel or a consonant.

6. Using an **if expression**, store "Pass" or "Fail" in a val (the pass mark is 40) and print it.

7. BMI: bmi = weight / (height * height), with weight in kg and height in metres. Print *Underweight* (below 18.5), *Normal* (below 25), *Overweight* (below 30) or *Obese*. Try weight 60 and height 1.65.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. If several conditions in an if / else if chain are true, which block runs?

Q2. When is else required with if?

Q3. What does in 1..10 mean?

Q4. How do you match two values in one when branch?

Q5. What happens if no when branch matches and there is no else?

WEEK 1 • DAY 5

Loops: Repeating Work

Today's goal: Repeat code with for, while and do-while loops, and control loops with break and continue.

1. Why loops?

Imagine printing the numbers 1 to 1000. Writing 1000 println lines is impossible. A **loop** repeats code for you.

2. Ranges

Range	Values	Meaning
1..5	1 2 3 4 5	from 1 to 5, **including** 5
1 until 5	1 2 3 4	from 1 up to 5, **excluding** 5
5 downTo 1	5 4 3 2 1	counting backwards
0..20 step 5	0 5 10 15 20	jump by 5

3. The for loop

for with ranges

```
fun main() {
    for (i in 1..5) {
        print("$i ")
    }
    println()

    for (i in 1 until 5) print("$i ")
    println()

    for (i in 10 downTo 1 step 3) print("$i ")
    println()

    for (i in 0..20 step 5) print("$i ")
    println()
}
```

Output

```
1 2 3 4 5
1 2 3 4
10 7 4 1
0 5 10 15 20
```

Read **for (i in 1..5)** as "for each **i** from 1 to 5". The variable **i** takes a new value each time the loop runs. If the body is only one line, you can leave out the { }.

Multiplication table

```
fun main() {
    val n = 7
    for (i in 1..10) {
        println("$n x $i = ${n * i}")
    }
}
```

Output

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Sum and factorial

```
fun main() {
    var sum = 0
    for (i in 1..100) {
        sum += i
    }
    println("Sum of 1 to 100 = $sum")

    var factorial = 1L          // Long, because factorials grow fast
    for (i in 1..10) {
        factorial *= i
    }
    println("10! = $factorial")
}
```

Output

```
Sum of 1 to 100 = 5050
10! = 3628800
```

4. Looping over a String

Each letter

```
fun main() {
    val word = "Android"
    for (letter in word) {
        println(letter)
    }
}
```

Output

```
A
n
d
r
o
i
d
```

Count the vowels

```
fun main() {
    val sentence = "I am learning Kotlin"
    var vowels = 0

    for (ch in sentence.lowercase()) {
        if (ch in "aeiou") {
            vowels++
        }
    }
    println("Number of vowels: $vowels")
}
```

Output

```
Number of vowels: 7
```

5. The while loop

while repeats as long as its condition is true. Use it when you don't know in advance how many times to loop.

Counting with while

```
fun main() {
    var count = 1
    while (count <= 5) {
        println("Count is $count")
        count++
    }
    println("Loop finished")
}
```

Output

```
Count is 1
Count is 2
Count is 3
Count is 4
Count is 5
Loop finished
```

Careful: If you forget **count++**, the condition stays true forever. That is an **infinite loop**. In IntelliJ, press the red Stop button to end it.

Reverse a number and add its digits

```
fun main() {
    var number = 12345
    var reversed = 0
    var digitSum = 0

    while (number > 0) {
        val digit = number % 10          // take the last digit
        reversed = reversed * 10 + digit
        digitSum += digit
        number /= 10                     // remove the last digit
    }
    println("Reversed: $reversed")
    println("Sum of digits: $digitSum")
}
```

Output

```
Reversed: 54321
Sum of digits: 15
```

Trace it on paper for a smaller number (123) to see how it works:

Round	number	digit	reversed	digitSum
start	123	-	0	0
1	12	3	3	3
2	1	2	32	5
3	0	1	321	6

6. do ... while

do-while checks the condition at the **end**, so the body always runs **at least once**.

do-while runs once

```
fun main() {
    var i = 10
    do {
        println("This runs at least once. i = $i")
        i++
    } while (i < 5)
}
```

Output

```
This runs at least once. i = 10
```

7. break and continue

Keyword	What it does
break	Stops the loop **completely**
continue	Skips the **rest of this round** and goes to the next one

break and continue

```
fun main() {
    for (i in 1..10) {
        if (i == 7) break           // stop when i is 7
        if (i % 2 == 0) continue    // skip even numbers
        print("$i ")
    }
    println()
}
```

Output

```
1 3 5
```

8. Nested loops: patterns

A loop inside another loop. The **outer** loop controls the rows and the **inner** loop controls the columns.

Star triangle

```
fun main() {
    val rows = 5
    for (i in 1..rows) {
        for (j in 1..i) {
            print("* ")
        }
        println()
    }
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

repeat and number triangle

```
fun main() {
    repeat(3) {
        println("Hello Kotlin!")
    }

    for (i in 1..4) {
        for (j in 1..i) print(j)
        println()
    }
}
```

Output

```
Hello Kotlin!
Hello Kotlin!
Hello Kotlin!
1
12
123
1234
```

Tip: repeat(n) { } is the simplest way to run code n times when you don't need a counter.

Practice Exercises

1. Print all the even numbers from 2 to 50 using **step**.

2. Print the multiplication table of 9 from 1 to 12.

3. Find the sum of all the odd numbers from 1 to 50.

4. Count how many times the letter 'a' appears in "banana".

5. Check whether 29 is a **prime number** (a number that can be divided only by 1 and itself).

6. Print the first 10 **Fibonacci** numbers: 0 1 1 2 3 5 8 13 21 34.

7. Print an **upside-down** star triangle with 5 rows.

8. Countdown: print 10 down to 1 using a while loop, then print "Happy New Year!".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between 1..5 and 1 until 5?

Q2. What is the difference between while and do-while?

Q3. What is the difference between break and continue?

Q4. What does for (i in 10 downTo 0 step 5) print("$i ") print?

Q5. What causes an infinite loop?

WEEK 1 • DAY 6

Functions + Weekly Project: Calculator

Today's goal: Write your own functions with parameters and return values, then build a complete calculator.

1. What is a function?

A function is a **named block of code** that does one job. You write it once and **call** (use) it as many times as you like. You've already used functions: **main()**, **println()** and **readln()**.

Your first function

```
fun greet() {
    println("Hello! Welcome to Kotlin.")
}

fun main() {
    greet()     // call the function
    greet()     // call it again
}
```

Output

```
Hello! Welcome to Kotlin.
Hello! Welcome to Kotlin.
```

2. Parameters: giving input to a function

Function with a parameter

```
fun greet(name: String) {
    println("Hello, $name!")
}

fun main() {
    greet("Anjana")
    greet("Rahul")
}
```

Output

```
Hello, Anjana!
Hello, Rahul!
```

name: String is a **parameter**: an input with a name and a type. "Anjana" is the **argument**: the actual value you pass in.

3. Return values: getting an answer back

Function that returns a value

```
fun add(a: Int, b: Int): Int {
    return a + b
}

fun main() {
    val result = add(10, 20)
    println("10 + 20 = $result")
    println("5 + 7 = ${add(5, 7)}")
}
```

Output

```
10 + 20 = 30
5 + 7 = 12
```

Part	Meaning
fun add	Function name
(a: Int, b: Int)	Two parameters, both Int
: Int	The **return type**: this function gives back an Int
return a + b	Sends the answer back to the caller

A function that doesn't return anything has the return type **Unit**. You don't need to write it.

4. Single-expression functions

If a function only returns one expression, you can write it on one line with **=**:

One-line functions

```
fun square(n: Int) = n * n
fun isEven(n: Int) = n % 2 == 0

fun main() {
    println(square(6))
    println(isEven(10))
    println(isEven(7))
}
```

Output

```
36
true
false
```

5. Default and named arguments

Default values

```
fun orderCoffee(size: String = "Medium", sugar: Int = 1) {
    println("Coffee: $size, sugar: $sugar spoon(s)")
}

fun main() {
    orderCoffee()                          // uses both defaults
    orderCoffee("Large")                   // sugar uses its default
    orderCoffee(sugar = 0)                 // named argument
    orderCoffee(sugar = 2, size = "Small") // any order with names
}
```

Output

```
Coffee: Medium, sugar: 1 spoon(s)
Coffee: Large, sugar: 1 spoon(s)
Coffee: Medium, sugar: 0 spoon(s)
Coffee: Small, sugar: 2 spoon(s)
```

Tip: Default and named arguments are used **everywhere** in Jetpack Compose (Android UI), so learn them well!

6. Functions working together

Report card with functions

```
fun calculateTotal(m1: Int, m2: Int, m3: Int): Int = m1 + m2 + m3

fun calculateAverage(total: Int, subjects: Int): Double = total.toDouble() / subjects

fun getGrade(average: Double): String {
    return when {
        average >= 90 -> "A+"
        average >= 80 -> "A"
        average >= 70 -> "B"
        average >= 40 -> "C"
        else -> "Fail"
    }
}

fun main() {
    val total = calculateTotal(85, 92, 78)
    val average = calculateAverage(total, 3)
    println("Total: $total")
    println("Average: $average")
    println("Grade: ${getGrade(average)}")
}
```

Output

```
Total: 255
Average: 85.0
Grade: A
```

Function with a loop

```
fun factorial(n: Int): Long {
    var result = 1L
    for (i in 1..n) {
        result *= i
    }
    return result
}

fun main() {
    for (n in 1..5) {
        println("$n! = ${factorial(n)}")
    }
}
```

Output

```
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

Tip: Good function names are **verbs** that describe the job: calculateTotal, getGrade, isEven. Each function should do **one** thing.

WEEKLY PROJECT: Console Calculator

Now combine **everything** from this week: variables, operators, when, loops and functions.

Version 1: works in the Playground

Calculator v1

```
fun add(a: Double, b: Double) = a + b
fun subtract(a: Double, b: Double) = a - b
fun multiply(a: Double, b: Double) = a * b

fun divide(a: Double, b: Double): String {
    return if (b == 0.0) "Error: cannot divide by zero" else (a / b).toString()
}

fun calculate(a: Double, op: Char, b: Double): String {
    return when (op) {
        '+' -> add(a, b).toString()
        '-' -> subtract(a, b).toString()
        '*' -> multiply(a, b).toString()
        '/' -> divide(a, b)
        else -> "Error: unknown operator $op"
    }
}

fun main() {
    println("12 + 8 = ${calculate(12.0, '+', 8.0)}")
    println("12 - 8 = ${calculate(12.0, '-', 8.0)}")
    println("12 * 8 = ${calculate(12.0, '*', 8.0)}")
    println("12 / 8 = ${calculate(12.0, '/', 8.0)}")
    println("12 / 0 = ${calculate(12.0, '/', 0.0)}")
    println("12 ^ 8 = ${calculate(12.0, '^', 8.0)}")
}
```

Output

```
12 + 8 = 20.0
12 - 8 = 4.0
12 * 8 = 96.0
12 / 8 = 1.5
12 / 0 = Error: cannot divide by zero
12 ^ 8 = Error: unknown operator ^
```

Version 2: interactive (IntelliJ)

Keep the same functions from Version 1, and replace main with this one. It keeps asking until the user types **q**.

Calculator v2 (main only) (run in IntelliJ: input typed by the user is shown in the output)

```
fun main() {
    println("=== Anjana's Calculator ===")
    while (true) {
        print("Enter first number (or q to quit): ")
        val first = readln()
        if (first == "q") break

        print("Enter operator (+ - * /): ")
        val opText = readln()
        val op = if (opText.isNotEmpty()) opText[0] else ' '

        print("Enter second number: ")
        val second = readln()

        val a = first.toDoubleOrNull()
        val b = second.toDoubleOrNull()
        if (a == null || b == null) {
            println("Please enter valid numbers!")
            println()
            continue
        }
        println("Result: ${calculate(a, op, b)}")
        println()
    }
    println("Goodbye!")
}
```

Output

```
=== Anjana's Calculator ===
Enter first number (or q to quit): 10
Enter operator (+ - * /): *
Enter second number: 5
Result: 50.0

Enter first number (or q to quit): abc
Enter operator (+ - * /): +
Enter second number: 2
Please enter valid numbers!

Enter first number (or q to quit): 9
Enter operator (+ - * /): /
Enter second number: 0
Result: Error: cannot divide by zero

Enter first number (or q to quit): q
Goodbye!
```

while (true) loops forever until **break** runs. **toDoubleOrNull()** gives null if the text is not a number, so the program doesn't crash.

Project challenges (try them!)

- Add the **%** (remainder) operator.
- Add a **^** (power) operator using pow from kotlin.math.
- Count how many calculations the user did and print the count before "Goodbye!".
- Upload your calculator to your **kotlin-practice** GitHub repository.

Week 1 checklist

I can...	Done?
Write and run a Kotlin program with main and println	[ ]
Choose correctly between val and var	[ ]
Use Int, Double, Boolean, Char and String	[ ]
Use string templates with $ and ${ }	[ ]
Use arithmetic, comparison and logical operators	[ ]
Convert types with toInt(), toDouble() and toString()	[ ]
Write if / else if / else and when	[ ]
Write for, while and do-while loops, and use break and continue	[ ]
Write functions with parameters, return values and default arguments	[ ]
Build the calculator project	[ ]

Quick Quiz

Q1. What keyword creates a function?

Q2. In fun add(a: Int, b: Int): Int, what does the last Int mean?

Q3. What is the return type of a function that returns nothing?

Q4. Rewrite fun double(x: Int): Int { return x * 2 } as a single-expression function.

Q5. Why do we use toDoubleOrNull() in the calculator instead of toDouble()?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Introduction & Your First Program

Exercise 1

```
fun main() {
    println("Anjana")
    println("ABC College")
    println("Kochi")
}
```

Output

```
Anjana
ABC College
Kochi
```

Exercise 2

```
fun main() {
    println("Anjana\nABC College\nKochi")
}
```

Output

```
Anjana
ABC College
Kochi
```

Exercise 3

```
fun main() {
    println("*")
    println("**")
    println("***")
    println("****")
}
```

Output

```
*
**
***
****
```

Exercise 4

```
fun main() {
    println("Anjana says \"I love coding!\"")
}
```

Output

```
Anjana says "I love coding!"
```

Exercise 5

```
fun main() {
    println("Subject\tMarks")
    println("Maths\t90")
    println("Physics\t85")
    println("English\t88")
}
```

Output

```
Subject Marks
Maths   90
Physics 85
English 88
```

Exercise 6

```
fun main() {
    // Print a top border
    println("===== My Profile =====")
    // Print my name
    println("Name    : Anjana")
    // Print a bottom border
    println("======================")
}
```

Output

```
===== My Profile =====
Name    : Anjana
======================
```

Quiz answers

Q1. In the main function: fun main().

Q2. println moves to a new line after printing; print stays on the same line.

Q3. \n

Q4. No. Kotlin is case-sensitive, and Println gives an error.

Q5. It ignores them. They are only for humans.

Day 2: Variables & Data Types

Exercise 1

```
fun main() {
    val name = "Anjana"
    val college = "ABC College"
    var semester = 1
    println("$name studies at $college, semester $semester")
    semester = 2
    println("Now in semester $semester")
}
```

Output

```
Anjana studies at ABC College, semester 1
Now in semester 2
```

Exercise 2

```
fun main() {
    val marks = 90
    val distance = 384_400_000L
    val height = 1.62
    val isHappy = true
    val initial = 'A'
    val city = "Kochi"
    println(marks)
    println(distance)
    println(height)
    println(isHappy)
    println(initial)
    println(city)
}
```

Output

```
90
384400000
1.62
true
A
Kochi
```

Exercise 3

```
fun main() {
    val name = "Anjana"
    val age = 20
    val city = "Kochi"
    println("Hello, my name is $name. I am $age years old and I live in $city.")
}
```

Output

```
Hello, my name is Anjana. I am 20 years old and I live in Kochi.
```

Exercise 4

```
fun main() {
    val word = "Android"
    println(word.length)
    println(word.uppercase())
    println(word[0])
    println(word[word.length - 1])
    println(word.reversed())
}
```

Output

```
7
ANDROID
A
d
diordnA
```

Exercise 5

```
// A val cannot be changed. Use var if the value must change:
fun main() {
    var pi = 3.14
    pi = 3.14159
    println(pi)
}
```

Output

```
3.14159
```

Exercise 6

```
fun main() {
    val title = "Kotlin for Beginners"
    val author = "J. Smith"
    val price = 499.0
    val pages = 320
    val isAvailable = true
    println("Title    : $title")
    println("Author   : $author")
    println("Price    : Rs $price")
    println("Pages    : $pages")
    println("Available: $isAvailable")
}
```

Output

```
Title    : Kotlin for Beginners
Author   : J. Smith
Price    : Rs 499.0
Pages    : 320
Available: true
```

Quiz answers

Q1. val cannot be changed after it is set; var can be changed.

Q2. val. Use var only when the value must change.

Q3. Double.

Q4. Char uses single quotes ('A'); String uses double quotes ("A").

Q5. Sum: 5

Q6. Long.

Day 3: Operators, Type Conversion & User Input

Exercise 1

```
fun main() {
    val a = 25
    val b = 4
    println(a + b)
    println(a - b)
    println(a * b)
    println(a / b)
    println(a % b)
    println(a.toDouble() / b)
}
```

Output

```
29
21
100
6
1
6.25
```

Exercise 2

```
fun main() {
    val average = (78 + 85 + 92) / 3.0
    println("Average = $average")
}
```

Output

```
Average = 85.0
```

Exercise 3

```
fun main() {
    val year = 2024
    println(year % 4 == 0)
}
```

Output

```
true
```

Exercise 4

```
fun main() {
    val km = 5
    println("Metres: ${km * 1000}")
    println("Miles: ${km * 0.621371}")
}
```

Output

```
Metres: 5000
Miles: 3.106855
```

Exercise 5 (IntelliJ)

```
fun main() {
    print("First number: ")
    val a = readln().toInt()
    print("Second number: ")
    val b = readln().toInt()
    println("Sum = ${a + b}")
}
```

Output

```
First number: 7
Second number: 8
Sum = 15
```

Exercise 6

```
fun main() {
    println(10 / 3)
    println(10 % 3)
    println(10.0 / 4)
    println("10" + 3)
    println(5 > 3 && 2 > 4)
}
```

Output

```
3
1
2.5
103
false
```

Quiz answers

Q1. The remainder of a division. For example, 17 % 5 is 2.

Q2. 4, because Int divided by Int gives an Int. Use 9.0 / 2 to get 4.5.

Q3. "45".toInt()

Q4. Only when both a and b are true.

Q5. Always a String.

Q6. = stores a value; == compares two values.

Day 4: Making Decisions: if, else and when

Exercise 1

```
fun main() {
    val n = -8
    if (n > 0) {
        println("Positive")
    } else if (n < 0) {
        println("Negative")
    } else {
        println("Zero")
    }
}
```

Output

```
Negative
```

Exercise 2

```
fun main() {
    for (year in listOf(2024, 1900, 2000)) {
        val leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0
        println("$year leap year? $leap")
    }
}
```

Output

```
2024 leap year? true
1900 leap year? false
2000 leap year? true
```

Exercise 3

```
fun main() {
    val units = 150
    val rate = if (units <= 100) 3 else if (units <= 200) 5 else 8
    println("Bill = Rs ${units * rate}")
}
```

Output

```
Bill = Rs 750
```

Exercise 4

```
fun main() {
    val month = 8
    val name = when (month) {
        1 -> "January"
        2 -> "February"
        3 -> "March"
        4 -> "April"
        5 -> "May"
        6 -> "June"
        7 -> "July"
        8 -> "August"
        9 -> "September"
        10 -> "October"
        11 -> "November"
        12 -> "December"
        else -> "Invalid month"
    }
    println(name)
}
```

Output

```
August
```

Exercise 5

```
fun main() {
    val letter = 'e'
    when (letter) {
        'a', 'e', 'i', 'o', 'u' -> println("$letter is a vowel")
        else -> println("$letter is a consonant")
    }
}
```

Output

```
e is a vowel
```

Exercise 6

```
fun main() {
    val marks = 35
    val result = if (marks >= 40) "Pass" else "Fail"
    println(result)
}
```

Output

```
Fail
```

Exercise 7

```
fun main() {
    val weight = 60.0
    val height = 1.65
    val bmi = weight / (height * height)
    val category = when {
        bmi < 18.5 -> "Underweight"
        bmi < 25 -> "Normal"
        bmi < 30 -> "Overweight"
        else -> "Obese"
    }
    println("BMI category: $category")
}
```

Output

```
BMI category: Normal
```

Quiz answers

Q1. Only the first one that is true.

Q2. When if is used as an expression, for example when its result is stored in a variable.

Q3. The value is between 1 and 10, including 1 and 10.

Q4. Separate them with a comma: 6, 7 -> ...

Q5. Nothing runs. But if when returns a value, else is required.

Day 5: Loops: Repeating Work

Exercise 1

```
fun main() {
    for (i in 2..50 step 2) print("$i ")
    println()
}
```

Output

```
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

Exercise 2

```
fun main() {
    for (i in 1..12) {
        println("9 x $i = ${9 * i}")
    }
}
```

Output

```
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
9 x 11 = 99
9 x 12 = 108
```

Exercise 3

```
fun main() {
    var sum = 0
    for (i in 1..50 step 2) {
        sum += i
    }
    println("Sum of odd numbers = $sum")
}
```

Output

```
Sum of odd numbers = 625
```

Exercise 4

```
fun main() {
    var count = 0
    for (ch in "banana") {
        if (ch == 'a') count++
    }
    println("'a' appears $count times")
}
```

Output

```
'a' appears 3 times
```

Exercise 5

```
fun main() {
    val n = 29
    var isPrime = n > 1
    for (i in 2 until n) {
        if (n % i == 0) {
            isPrime = false
            break
        }
    }
    println("$n is prime? $isPrime")
}
```

Output

```
29 is prime? true
```

Exercise 6

```
fun main() {
    var a = 0
    var b = 1
    repeat(10) {
        print("$a ")
        val next = a + b
        a = b
        b = next
    }
    println()
}
```

Output

```
0 1 1 2 3 5 8 13 21 34
```

Exercise 7

```
fun main() {
    for (i in 5 downTo 1) {
        for (j in 1..i) print("* ")
        println()
    }
}
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Exercise 8

```
fun main() {
    var n = 10
    while (n >= 1) {
        println(n)
        n--
    }
    println("Happy New Year!")
}
```

Output

```
10
9
8
7
6
5
4
3
2
1
Happy New Year!
```

Quiz answers

Q1. 1..5 includes 5; 1 until 5 stops at 4.

Q2. while checks the condition first; do-while runs the body once before it checks.

Q3. break stops the whole loop; continue skips only the current round.

Q4. 10 5 0

Q5. A condition that never becomes false, for example forgetting to update the counter.

Day 6: Functions + Weekly Project: Calculator

Quiz answers

Q1. fun

Q2. The return type: the function returns an Int.

Q3. Unit (you don't have to write it).

Q4. fun double(x: Int) = x * 2

Q5. So the program doesn't crash when the user types something that is not a number.

Next: When you finish this week, tell your mentor and ask for the **Week 2** material. Great work, Anjana!